Garbage Collection in C#.Net



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



Garbage Collector

Generations in Garbage Collection

How to create Quickly Disposable Objects?

Garbage Collector

Garbage Collection in C#.Net

What is Garbage Collection?

- Automatic memory management is made possible by **Garbage Collection in .NET Framework**.
- When a class object is created at runtime, certain memory space is allocated to it in the heap memory.
- However, after all the actions related to the object are completed in the program, the memory space allocated to it is a waste as it cannot be used. In this case, garbage collection is very useful as it automatically releases the memory space after it is no longer required.

Garbage Collection occurs if at least one of multiple conditions is satisfied.

- If the system has low physical memory, then garbage collection is necessary.
- If the memory allocated to various objects in the heap memory exceeds a pre-set threshold, then garbage collection occurs.
- If the GC.Collect method is called, then garbage collection occurs. However, this method is only called under unusual situations as normally garbage collector runs automatically.

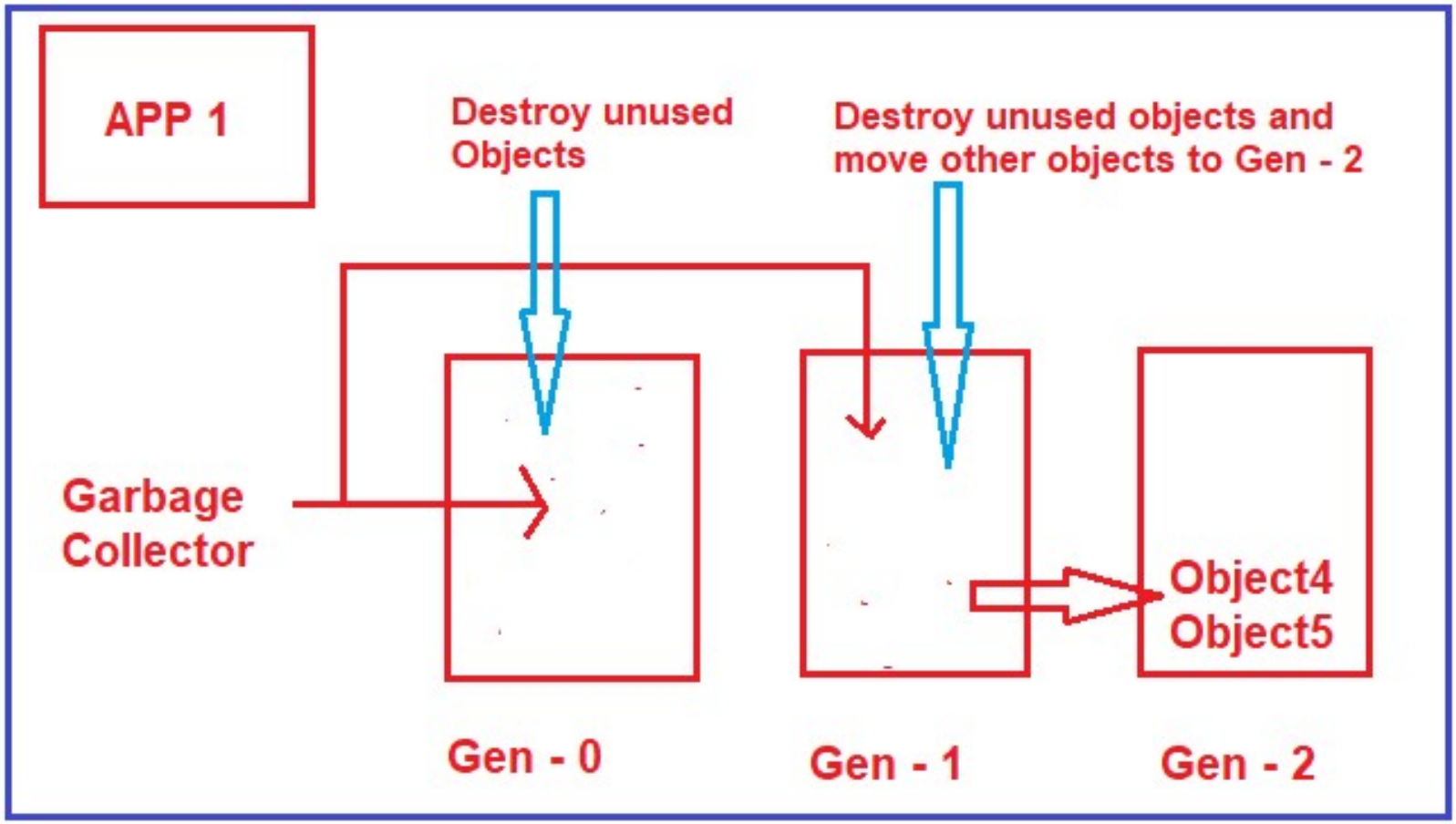
Generations of Garbage Collector

Garbage Collection in C#.Net

Generation 0 : All the **short-lived objects** such as temporary variables are contained in the generation 0 of the heap memory. All the newly allocated objects are also generation 0 objects implicitly unless they are large objects. In general, the frequency of garbage collection is the highest in generation 0.

Generation 1 : If space occupied by some generation 0 objects that are not released in a garbage collection run, then these objects get moved to generation 1. The objects in this generation are a sort of buffer between the short-lived objects in generation 0 and the long-lived objects in generation 2.

Generation 2 : If space occupied by some generation 1 objects that are not released in the next garbage collection run, then these objects get moved to generation 2. The objects in generation 2 are long lived such as static objects as they remain in the heap memory for the whole process duration.


APP 1
Destroy unused Objects
Destroy unused objects and move other objects to Gen - 2
Garbage Collector
Object4
Object5
Gen - 0
Gen - 1
Gen - 2

How to create quickly disposable object?

Garbage Collection in C#.Net

IDisposable

- IDisposable is an interface in C# that provides a mechanism for releasing unmanaged resources.
- It is typically used to clean up resources such as file handles, network connections, database connections, or other limited resources that need to be explicitly released.
- The IDisposable interface defines a single method called Dispose(), which is responsible for releasing the allocated resources.
- When an object implements IDisposable, it indicates that it requires explicit cleanup before it goes out of scope or is no longer needed.

```
public class MyResource : IDisposable
{
    private bool disposed = false;

    // Constructor
    public MyResource()
    {
        // Acquire unmanaged resources here
    }
    // Method that uses the resource
    public void DoSomething()
    {
        // Check if the object has been disposed
        if (disposed)
        {
            throw new ObjectDisposedException("MyResource", "The object has already been disposed.");
        }
        // Use the resource here
        Console.WriteLine("Doing something with MyResource...");
    }
    // Dispose method implementation
    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }
```

```
    // Protected Dispose method to be overridden by derived classes
    protected virtual void Dispose(bool disposing)
    {
        if (!disposed)
        {
            if (disposing)
            {
                // Dispose managed resources here
            }

            // Release unmanaged resources here

            disposed = true;
        }
    }

    // Finalizer (destructor)
    ~MyResource()
    {
        Dispose(false);
    }
}
```

References

https://www.geeksforgeeks.org/garbage-collection-in-c-sharp-dot-net-framework/

https://learn.microsoft.com/en-us/dotnet/standard/garbage-collection/fundamentals

https://www.geeksforgeeks.org/garbage-collection-in-c-sharp-dot-net-framework/

https://www.c-sharpcorner.com/UploadFile/skumaar_mca/garbage-collection-in-net-framework/

https://www.tutorialspoint.com/What-is-garbage-collection-in-Chash

https://www.codingame.com/playgrounds/6179/garbage-collection-and-c

https://dotnettutorials.net/lesson/garbage-collector/